

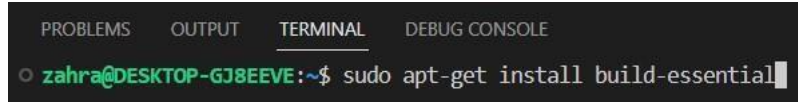
Installing SystemC on vscode

Linux:

If you already have configured your vscode to WSL, you should proceed as the following:

- Run the following command from your terminal or console to ensure you have all the necessary tools to compile SystemC files in your main folder.

```
$ sudo apt-get install build-essential
```



```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE
○ zahra@DESKTOP-GJ8EEVE:~$ sudo apt-get install build-essential
```

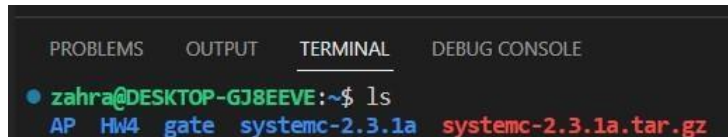
- Download the SystemC-2.3.1a by clicking [here](#).

If you want to download other versions, [click here](#).

- The downloaded file is named **systemc-2.3.1a.tar.gz**, unzip it using the following command. Make sure you are in the same directory as the downloaded file, in our case, you should copy-paste it in your home folder of Ubuntu drive and then unzip it.

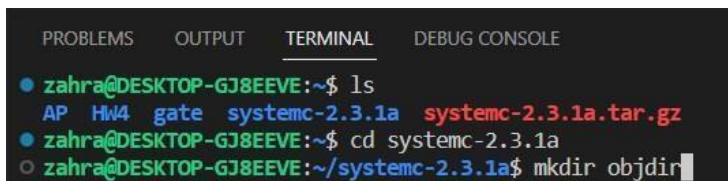
```
$ tar -xvf systemc-2.3.1a.tar.gz
```

Then you must have **systemc-2.3.1a** in your folder.



```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE
● zahra@DESKTOP-GJ8EEVE:~$ ls
AP HW4 gate systemc-2.3.1a systemc-2.3.1a.tar.gz
```

- Next, change the directory to the one that was just created by the tar command, and create a directory called objdir there.

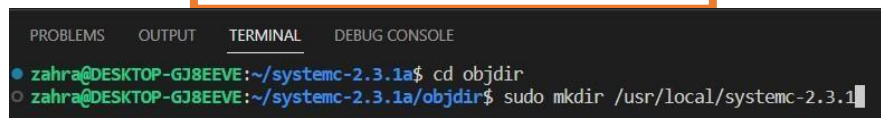


```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE
● zahra@DESKTOP-GJ8EEVE:~$ ls
AP HW4 gate systemc-2.3.1a systemc-2.3.1a.tar.gz
● zahra@DESKTOP-GJ8EEVE:~$ cd systemc-2.3.1a
○ zahra@DESKTOP-GJ8EEVE:~/systemc-2.3.1a$ mkdir objdir
```

- Go to the created directory and type the below command.

Notice you will need to use **sudo** to be able to create a directory in that folder.

```
$ sudo mkdir /usr/local/systemc-2.3.1
```



```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE
● zahra@DESKTOP-GJ8EEVE:~/systemc-2.3.1a$ cd objdir
○ zahra@DESKTOP-GJ8EEVE:~/systemc-2.3.1a/objdir$ sudo mkdir /usr/local/systemc-2.3.1
```

- After that, issue the following command.

```
$ sudo ../configure --prefix=/usr/local/systemc-2.3.1/
```

You should see an output like this:

```
-----
Configuration summary of SystemC 2.3.1 for x86_64-unknown-linux-gnu
-----

Directory setup (based on classic layout):
  Installation prefix (aka SYSTEMC_HOME):
    /usr/local/systemc-2.3.1
  Header files : <SYSTEMC_HOME>/include
  Libraries    : <SYSTEMC_HOME>/lib-linux64
  Documentation : <SYSTEMC_HOME>/docs
  Examples     : <SYSTEMC_HOME>/examples

Architecture : linux64
Compiler (flags): g++

Build settings:
  Enable compiler optimizations : yes
  Include debugging symbols     : no
  Coroutine package for processes: QuickThreads
  Disable async_request_update  : no
  Phase callbacks (experimental) : no
  Additional settings          :

-----
```

- Then write the following command, this part may take some time.

\$ sudo make

```
PROBLEMS  OUTPUT  TERMINAL  DEBUG CONSOLE
○ zahra@DESKTOP-GJ8EEVE:~/systemc-2.3.1a/objdir$ sudo make
```

- Next command is the following.

\$ sudo make install

```
PROBLEMS  OUTPUT  TERMINAL  DEBUG CONSOLE
○ zahra@DESKTOP-GJ8EEVE:~/systemc-2.3.1a/objdir$ sudo make install
```

Once all the installations are finished, you'll still be in /objdir directory, it'll be better to make a folder for your projects and homework in the main folder rather than installation directories.

All you must do to be able to access SystemC in other directories is to type the command below, each time you open your vscode.

\$ export SYSTEMC_HOME=/usr/local/systemc-2.3.1/

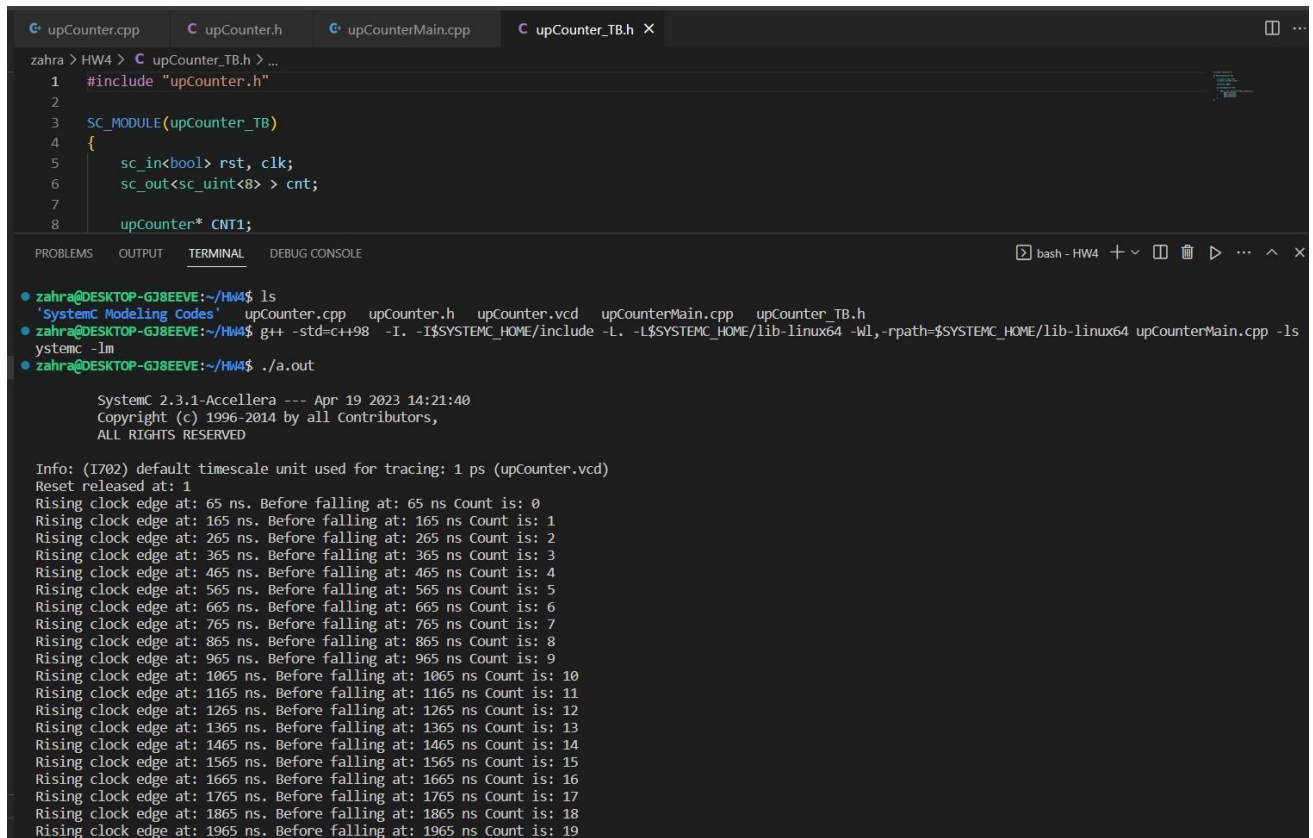
```
PROBLEMS  OUTPUT  TERMINAL  DEBUG CONSOLE
● zahra@DESKTOP-GJ8EEVE:~/systemc-2.3.1a/objdir$ cd ../../
● zahra@DESKTOP-GJ8EEVE:~$ ls
AP HW4 gate systemc-2.3.1a systemc-2.3.1a.tar.gz
● zahra@DESKTOP-GJ8EEVE:~$ cd HW4
● zahra@DESKTOP-GJ8EEVE:~/HW4$ export SYSTEMC_HOME=/usr/local/systemc-2.3.1/
○ zahra@DESKTOP-GJ8EEVE:~/HW4$
```

Now we'll demonstrate running a systemC code, "UpCounter", using the following command for building cpp files that use systemc:

\$g++ -std=c++98 -I. -I\$SYSTEMC_HOME/include -L. -L\$SYSTEMC_HOME/lib-linux64 -Wl,-rpath=\$SYSTEMC_HOME/lib-linux64 upCounterMain.cpp -lsystemc -lm

The text that was colored red, is supposed to be the name of the cpp file that you want to compile, in this case, we want to compile a cpp file called "upCounterMain.cpp" so we'll write it there.

As you can see the code compiles and runs correctly.



The screenshot displays a SystemC IDE interface with four tabs: `upCounter.cpp`, `upCounter.h`, `upCounterMain.cpp`, and `upCounter_TB.h`. The `upCounter_TB.h` tab is active, showing the following code:

```
1 #include "upCounter.h"
2
3 SC_MODULE(upCounter_TB)
4 {
5     sc_in<bool> rst, clk;
6     sc_out<sc_uint<8> > cnt;
7
8     upCounter* CNT1;
```

Below the code editor, the **TERMINAL** tab is selected, showing the execution output. The output includes the compilation command, SystemC version information, and a detailed timing trace of the counter's operation.

```
zahra@DESKTOP-GJ8EEVE:~/HW4$ ls
'SystemC Modeling Codes' upCounter.cpp upCounter.h upCounter.vcd upCounterMain.cpp upCounter_TB.h
zahra@DESKTOP-GJ8EEVE:~/HW4$ g++ -std=c++98 -I. -I$SYSTEMC_HOME/include -L. -L$SYSTEMC_HOME/lib-linux64 -Wl,-rpath=$SYSTEMC_HOME/lib-linux64 upCounterMain.cpp -lsystemc -lm
zahra@DESKTOP-GJ8EEVE:~/HW4$ ./a.out

SystemC 2.3.1-Accellera --- Apr 19 2023 14:21:40
Copyright (c) 1996-2014 by all Contributors,
ALL RIGHTS RESERVED

Info: (I702) default timescale unit used for tracing: 1 ps (upCounter.vcd)
Reset released at: 1
Rising clock edge at: 65 ns. Before falling at: 65 ns Count is: 0
Rising clock edge at: 165 ns. Before falling at: 165 ns Count is: 1
Rising clock edge at: 265 ns. Before falling at: 265 ns Count is: 2
Rising clock edge at: 365 ns. Before falling at: 365 ns Count is: 3
Rising clock edge at: 465 ns. Before falling at: 465 ns Count is: 4
Rising clock edge at: 565 ns. Before falling at: 565 ns Count is: 5
Rising clock edge at: 665 ns. Before falling at: 665 ns Count is: 6
Rising clock edge at: 765 ns. Before falling at: 765 ns Count is: 7
Rising clock edge at: 865 ns. Before falling at: 865 ns Count is: 8
Rising clock edge at: 965 ns. Before falling at: 965 ns Count is: 9
Rising clock edge at: 1065 ns. Before falling at: 1065 ns Count is: 10
Rising clock edge at: 1165 ns. Before falling at: 1165 ns Count is: 11
Rising clock edge at: 1265 ns. Before falling at: 1265 ns Count is: 12
Rising clock edge at: 1365 ns. Before falling at: 1365 ns Count is: 13
Rising clock edge at: 1465 ns. Before falling at: 1465 ns Count is: 14
Rising clock edge at: 1565 ns. Before falling at: 1565 ns Count is: 15
Rising clock edge at: 1665 ns. Before falling at: 1665 ns Count is: 16
Rising clock edge at: 1765 ns. Before falling at: 1765 ns Count is: 17
Rising clock edge at: 1865 ns. Before falling at: 1865 ns Count is: 18
Rising clock edge at: 1965 ns. Before falling at: 1965 ns Count is: 19
```

Windows GCC:

Prerequisite:

Step1: Install MinGW via MSYS2. ([link](#))

MSYS2 is a better way to get MinGW and its tools.

After installation, run the following commands in the MSYS2 shell to install MinGW:

```
pacman -Syu      # Updates package database
pacman -S mingw-w64-ucrt-x86_64-gcc # Install GCC for MinGW
pacman -S mingw-w64-x86_64-make # Install Make
gcc --version
```

If you installed MinGW or MSYS2, you must add the bin directory to your system's PATH variable.

For MSYS2, the mingw32-make binary is located in the C:\msys64\mingw64\bin directory (or mingw32\bin if you installed the 32-bit version).

To add it to the PATH:

1. Open System Properties → Advanced → Environment Variables.
2. Under System variables, find Path and click Edit.
3. Add the path to the bin directory (e.g., C:\msys64\mingw64\bin).

Step 2: Check mingw32-make Command

After setting up MinGW or MSYS2, verify that mingw32-make is accessible from the command prompt:

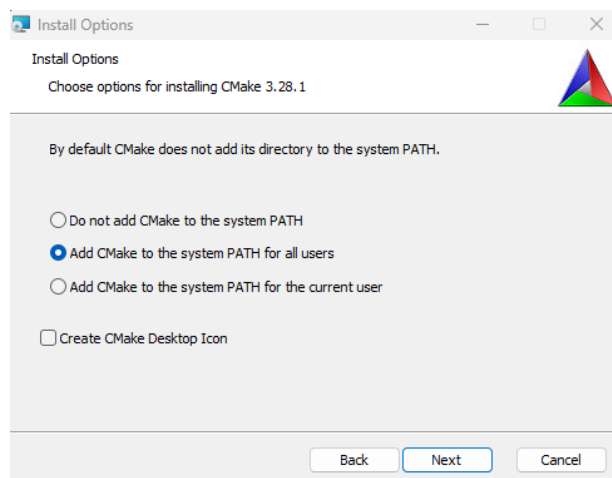
1. Open a new Command Prompt or MSYS2 terminal.
2. Type mingw32-make --version to confirm it's installed.

If you see a version number for mingw32-make, it means the tool is installed correctly.

Step 3: Download and install CMake

Download from this [link](#) .Run the installer.

During installation, ensure you select the option to Add CMake to the system PATH for all users. This step is critical to making the cmake command available in the command prompt



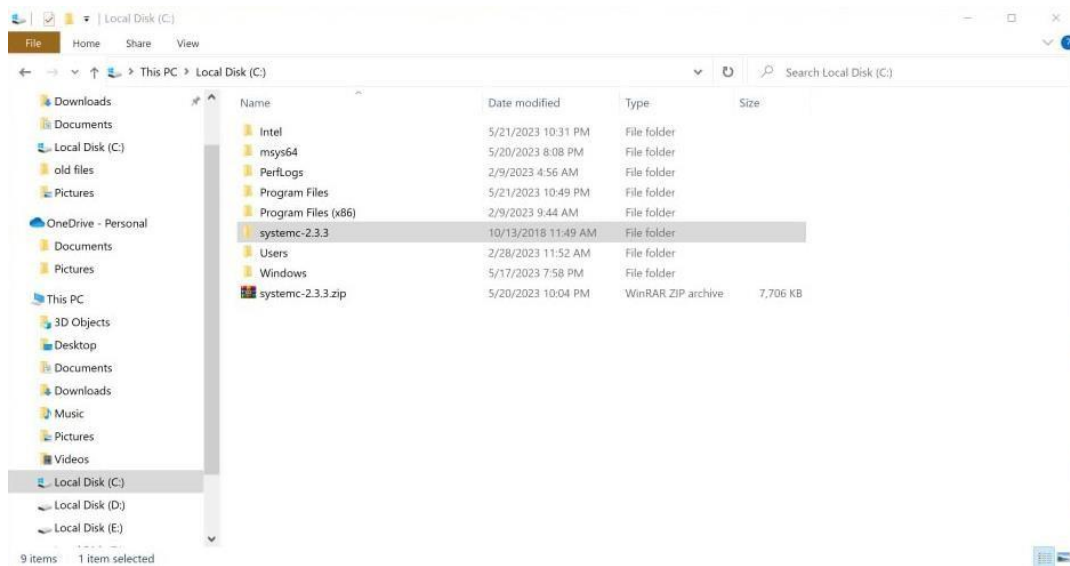
Run this on md:

```
cmake --version
```

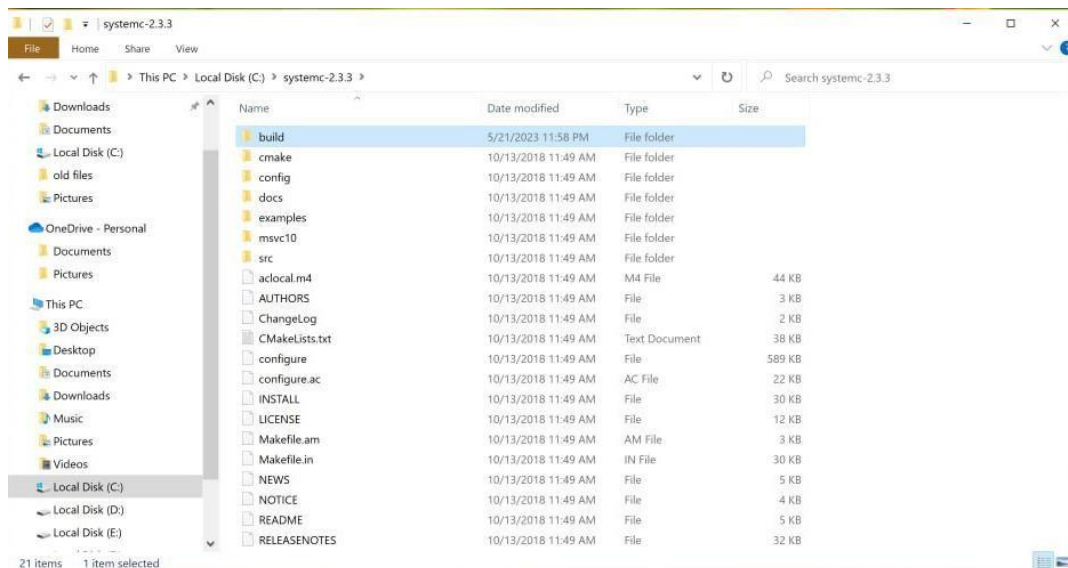
This works in systemC 2.3.3 version and higher therefore, ensure it is not the previous versions.

The compiler is c++98

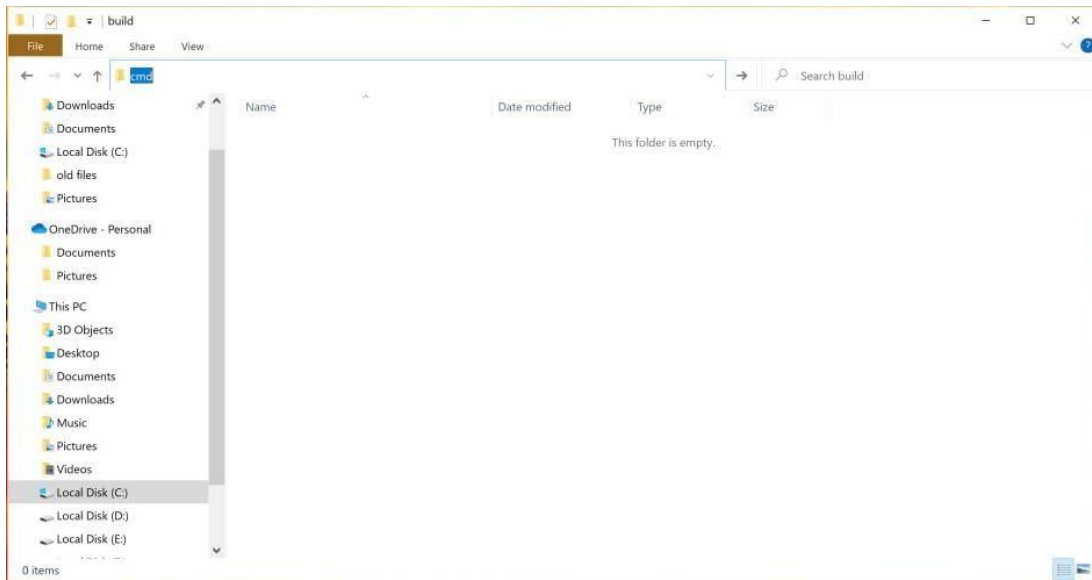
- in the first step you must extract the systemc-2.3.3 manually in your drive c.



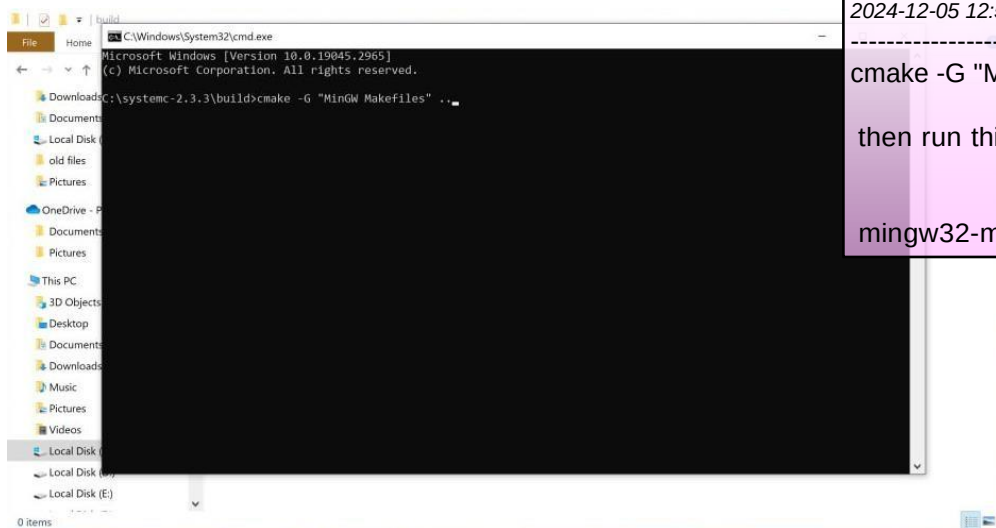
- make a folder named “build”



- open the build folder through cmd like the picture below, or else you could type the command “`cd C:\systemc-2.3.3\build`” in the cmd window.



- type the command “`C:\systemc-2.3.3\build> cmake -G “MinGW Makefiles”`” in the command prompt



Hani

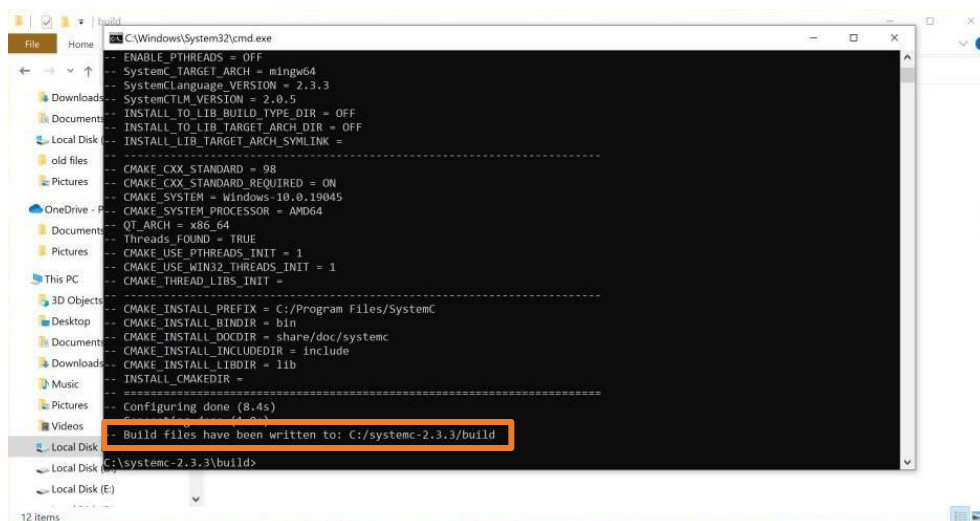
2024-12-05 12:54:19

`cmake -G "MinGW Makefiles" ..`

then run this:

`mingw32-make`

You get a similar output as shown below (the command results in the writing of the related build files in the build folder)



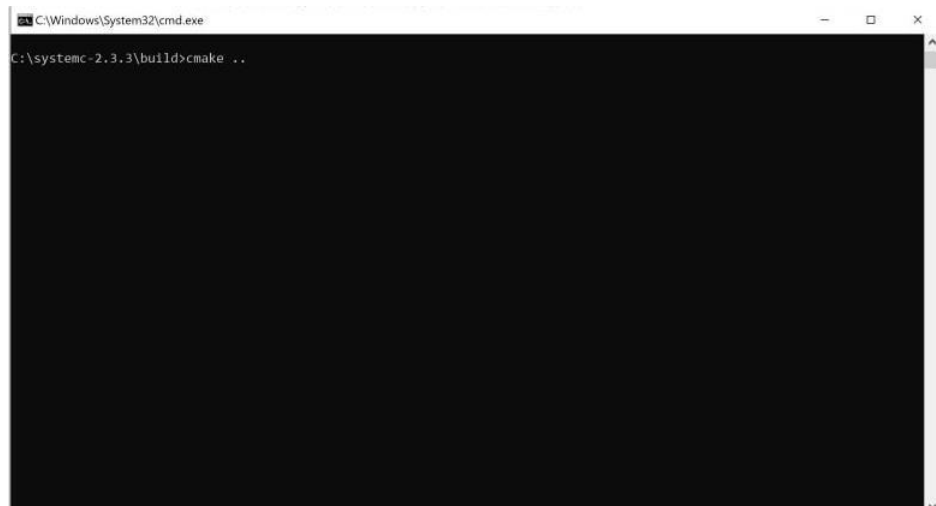
- **Verify the Build Process**

`cd C:/systemc-2.3.3/build`

`cmake -G "MinGW Makefiles" ..`

`mingw32-make`

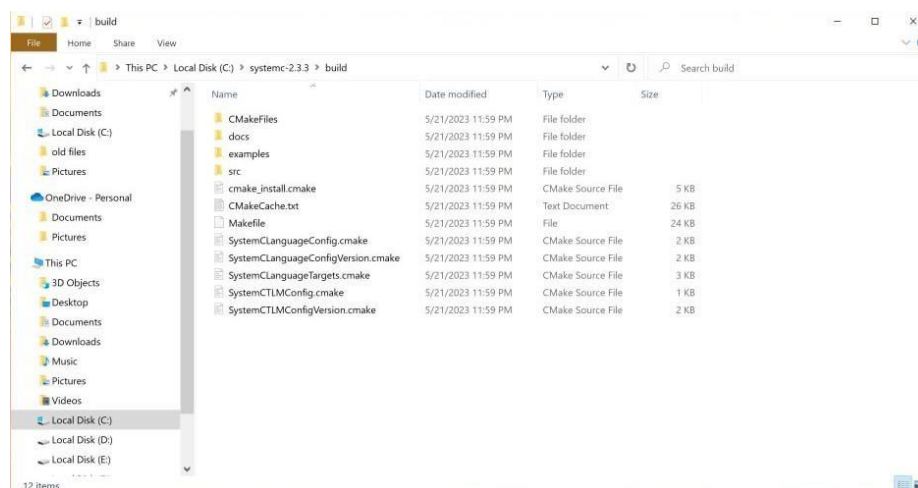
- issue the “cmake” command in the build folder in the cmd



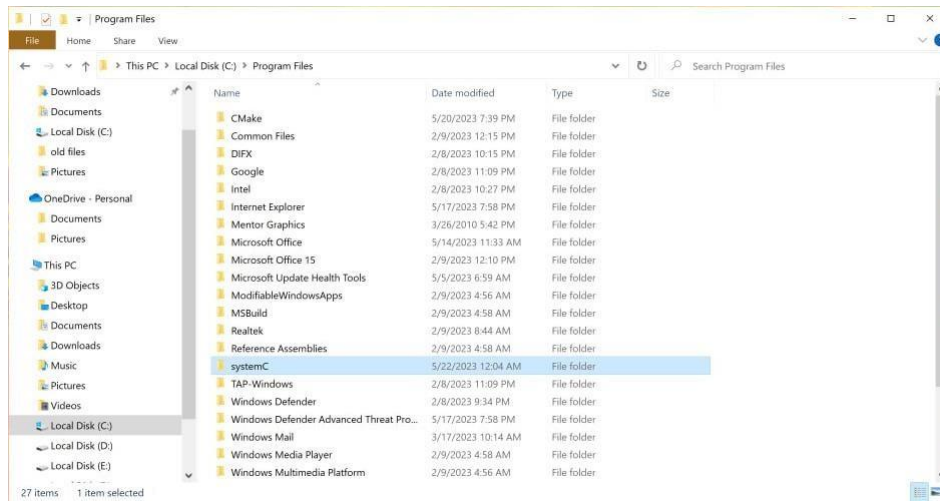
- type the “cmake --build” in the cmd



- the initial build folder should be looking like this after all these steps



- make a folder named “systemC” in your preferred directory, in this example, it had been made in C:\Program Files.



- copy the directory through the bar and then open the cmd as administrator, go to the build folder through “**cd C:\systemC-2.3.3\build**” command and type < **cmake --install . --prefix=”your folder address”**>

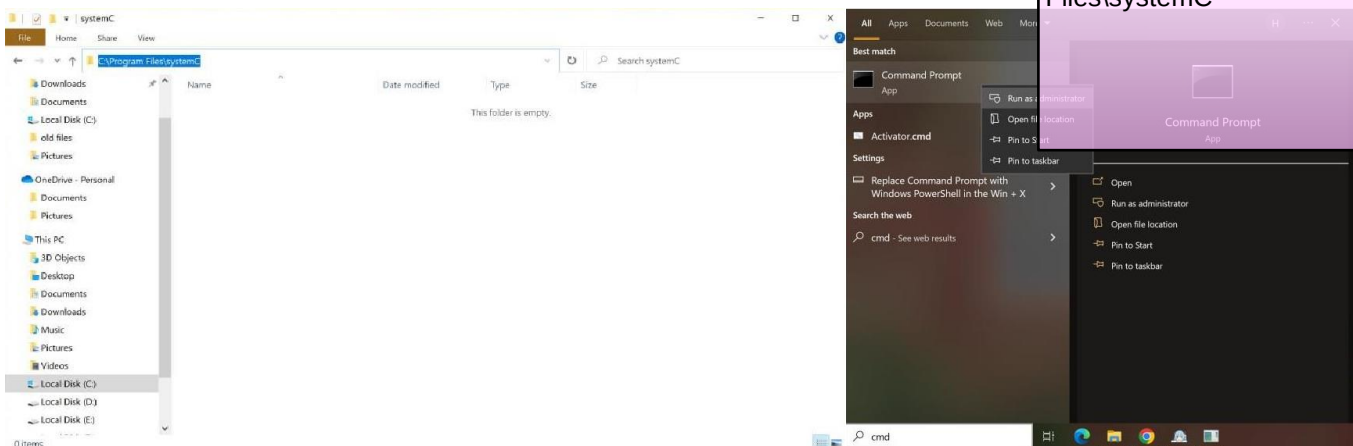
Make sure to use **”** and not **“**.

Your folder address is the systemC folder address that you copied earlier in this step.

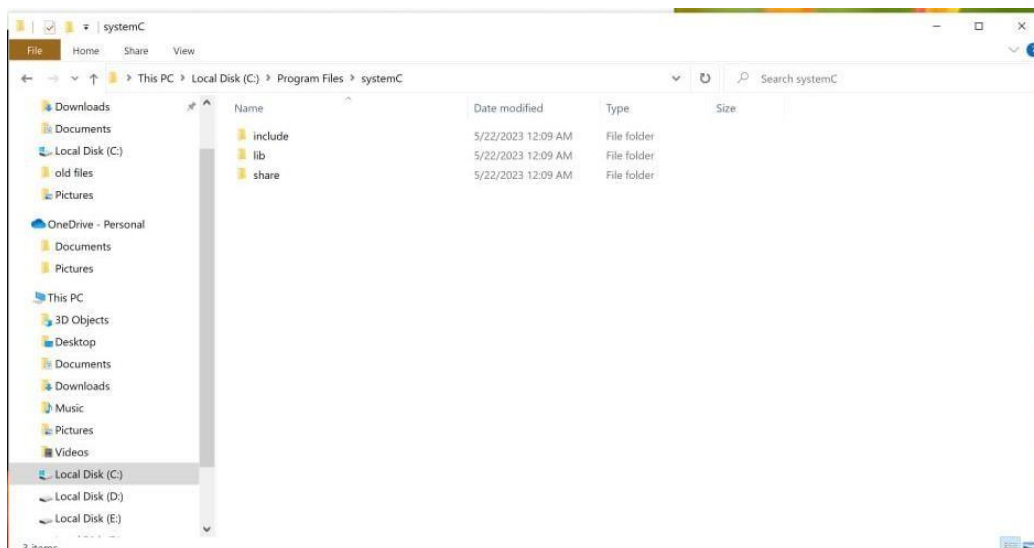
Hani

2024-12-05 12:33:38

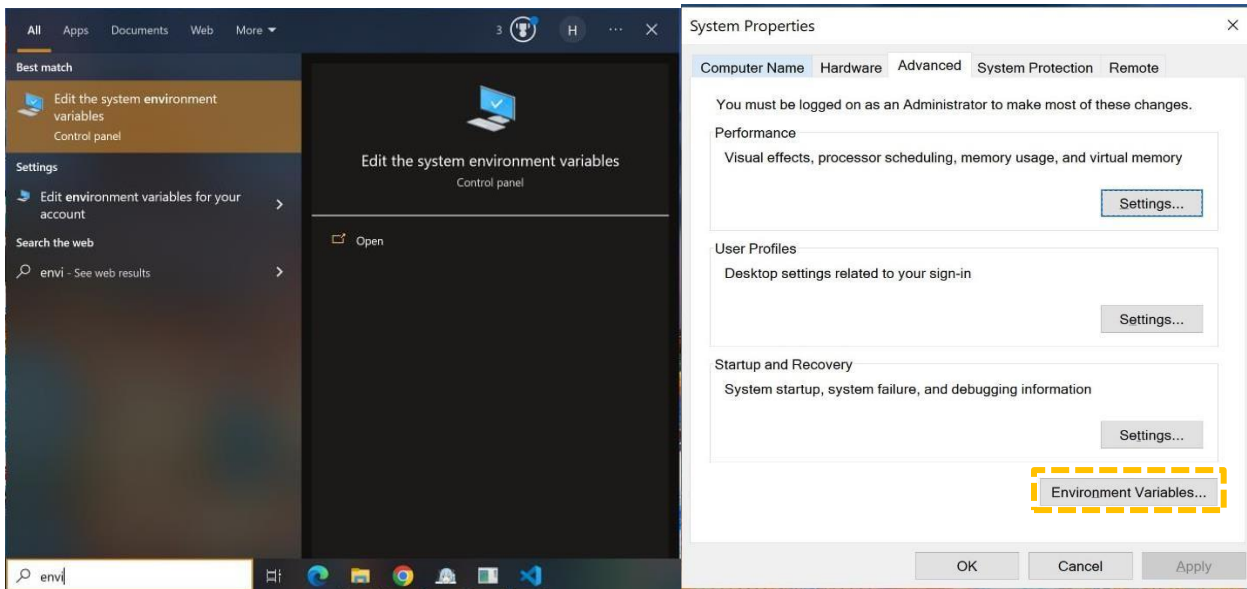
cmake --install . --prefix="C:\Program Files\systemC"



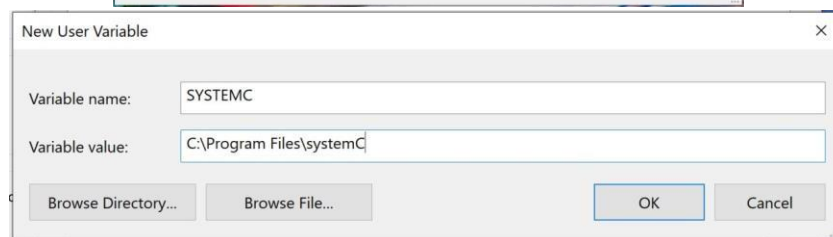
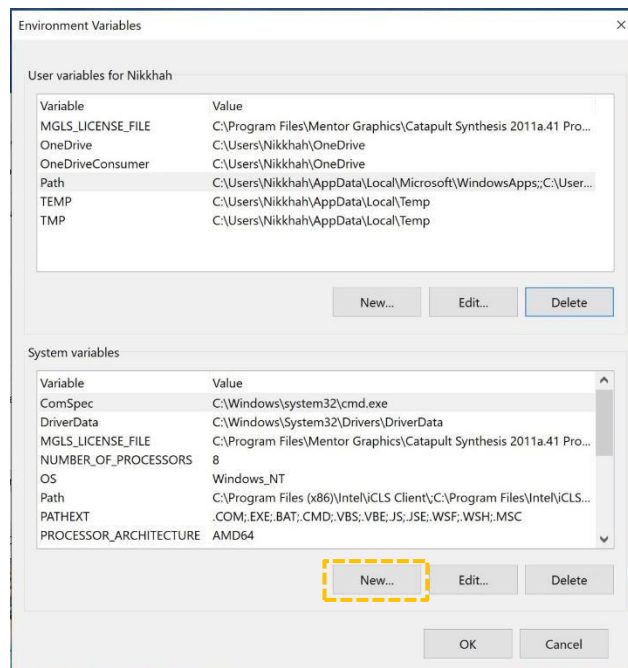
- the results will be as such:



- open the system environment to add new variables

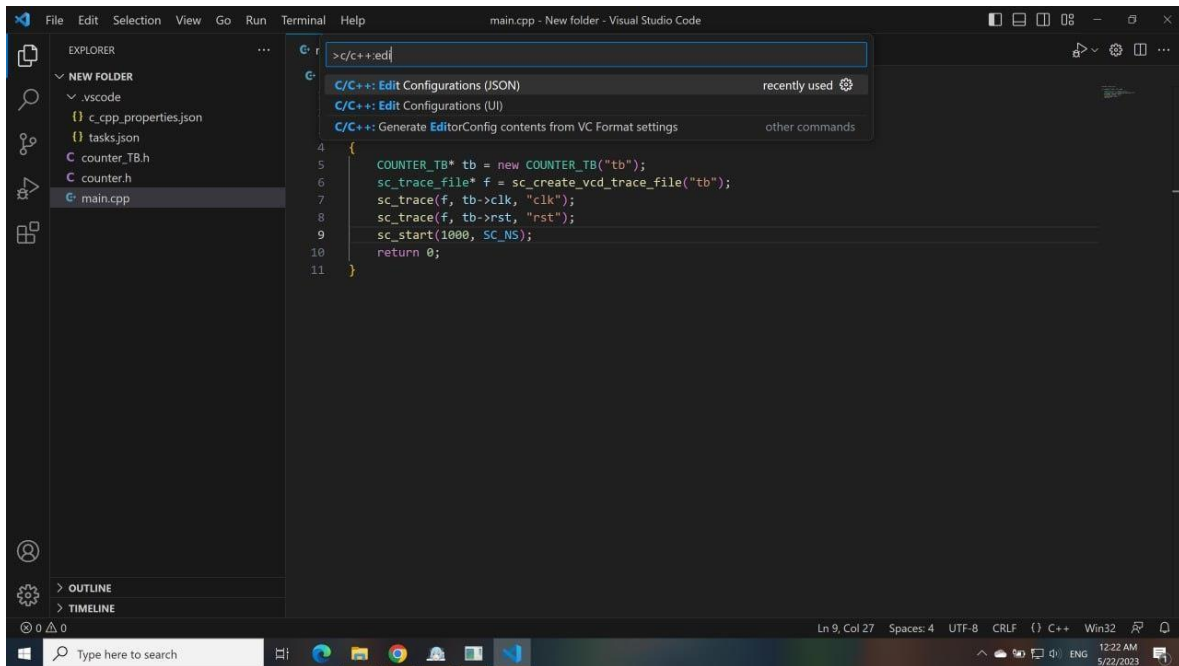


- in the “System variables” click in new button to create new variables, then write the “systemC” folder path Variable value section, Variable name is “SYSTEMC”



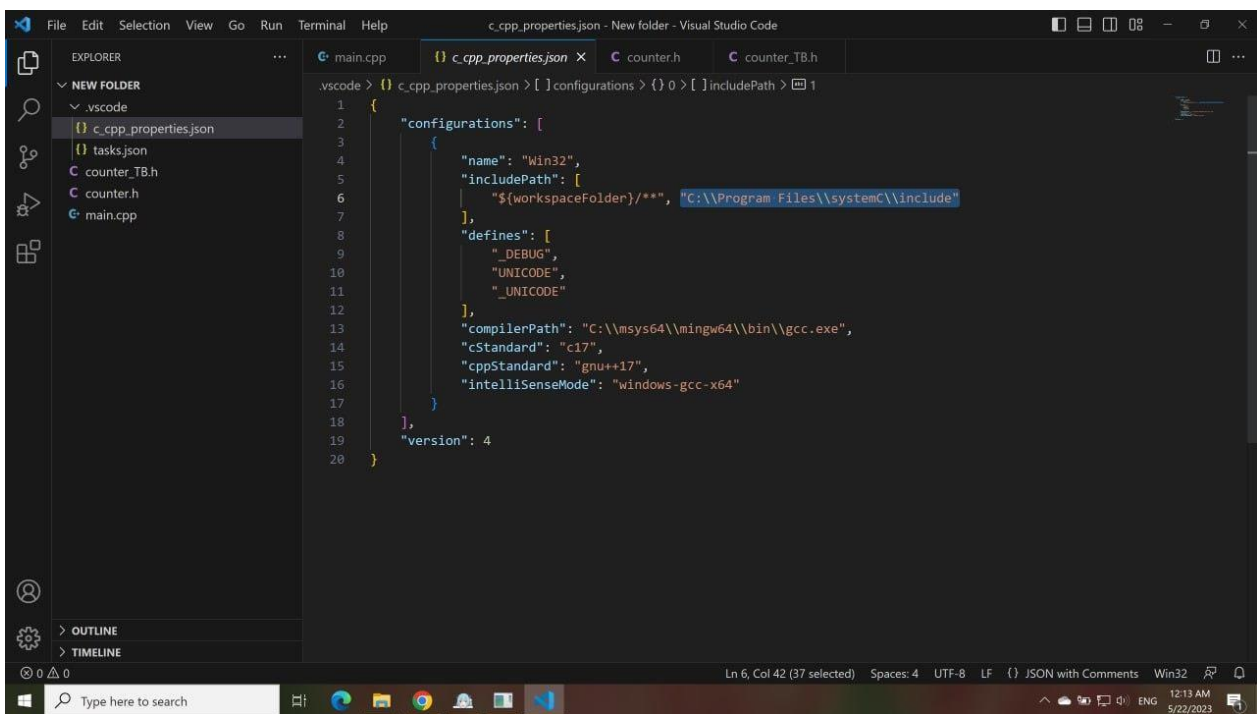
Vscode step up:

- in the next step you must open a folder with your systemC codes, then open vscode and use use “ctrl + shift + p” to open the bar, choose C/C++: edit configuration(JSON)



- change the source of “includePath” in “c_cpp_properties.json” to where your “systemC” folder is located (in the previous steps, you made this folder in your preferred directory)

make sure to put a double slash (\\) while writing the path.



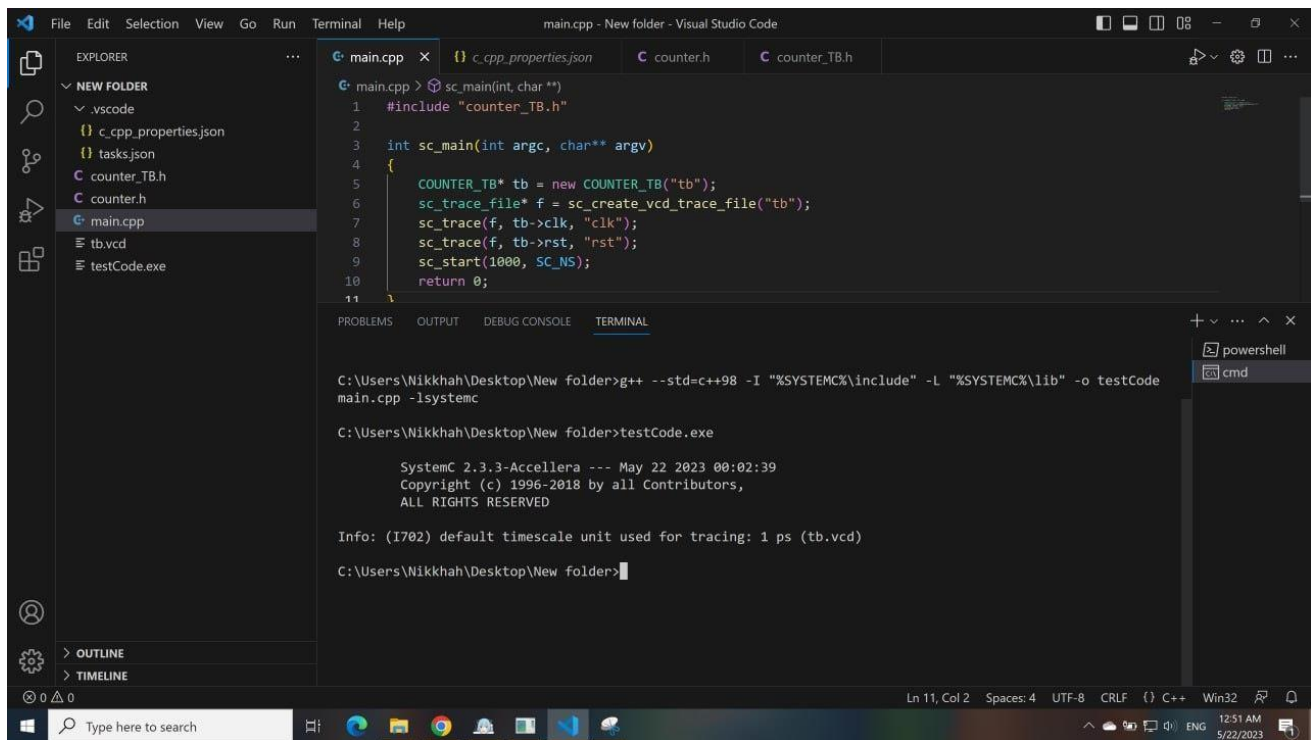
- compile your file by writing the following command in the terminal. (change the terminal to cmd instead of PowerShell)

```
g++ --std=c++98 -I "%SYSTEMC%\include" -L "%SYSTEMC%\lib" -o [output file name] [source files]
lsystemc
```

Hani

2024-12-05 13:08:41

```
g++ --std=c++98 -I
"%SYSTEMC%\include" -L
"%SYSTEMC%\lib" -o testCode
upCounterMain.cpp upCounter.cpp -
lsystemc
```



Note: Make sure that for `sc_lv<>` you have to insert extra space or your compiler takes it as `>>` operand.

Wrong form:

```
sc_signal    <sc_lv<32>>>    SI;
```

Correct form:

```
sc_signal    <sc_lv<32> >    SI;
```